Big Data and NoSQL - BookScope

ANDRÉ Mathis, VIALE Jean-Jacques

Données : lecteurs 20000 à 29999

1 128 761
notes réelles

10 000
lecteurs, sans biais de thème

10 000
livres (catalogue complet)

34 252
tags

Découpage déterministe, fait en amont



80 % base (903 033) · 10 % lot F6 · 10 % éval. E3

- Livres : médiane 35 notes, max 3 998 (The Hunger Games)
- Top 100 = 18,8 % des notes ; top 10 % = 56,3 %
- Lecteurs : médiane 90 notes (de 18 à 159)
- « Aimé » = note ≥ 4
- Nouveau lecteur simulé : --truncate k

Une collection par fichier source

Collection	Clé / identité	Rôle
ratings	(user_id, book_id) unique	903 033 notes
to_read	user_id, book_id	182 725 entrées
books	book_id + goodreads_book_id	pont d'identifiants
book_tags	goodreads_book_id, tag_id	comptes agrégés
tags	tag_id	34 252 noms

Trois accès fréquents

« Qui a aimé ce livre ? »
F1, F4, F5, index par **book_id**

« Qu'ont aimé ces lecteurs ? »
F1, F4, F5, index par **user_id**

« Les notes de ce lecteur ? »
F2, F3, index par **user_id**

F1 - Explorer un livre

Score thème
Tags pondérés par rareté ; ignorés s'ils sont sur plus de 50 % des livres

Score lecteurs
Cosinus sur les lecteurs communs (≥ 3), un gros lecteur pèse moins

Lift = observé / attendu
Proche de 1 : lien expliqué par la seule popularité

Score final
Moyenne des deux, +0,3 même auteur, +0,1 année proche ; même série en tête

The Hunger Games (livre 1) · 3 360 lecteurs

Livre lié	Preuve affichée
Catching Fire	même série · 1 516 lecteurs, ×2,3
Divergent	765 lecteurs, ×1,8 · « ya-dystopian »
Harry Potter 1	thème 0,27 · 1 860 lecteurs, ×1,7

F2 - Comparer deux lecteurs

Livres en commun

Écart de note	Similarité
0	100 %
1	75 %
2	50 %
3	25 %
4	0 %

Moyenne sur les livres communs

Genres aimés

Genres définis dans config.py (fantasy, romance, horror…).

Chaque livre noté ajoute **(note − 3) / 2** à ses genres.

Vecteurs comparés par **cosinus** : l'orientation compte, pas le volume.

F3 - Recommander un livre

10 graines
livres que le lecteur a aimés

→

F1 sur chacune
10 voisins par graine

→

Vote
+1 par apparition, puis Σ scores F1, puis id

- **Graines :** les 10 livres les mieux notés du lecteur (note ≥ 4), les plus récents en cas d'égalité.
- **Candidats :** pour chaque graine, les 10 livres les plus proches selon F1, hors livres déjà notés.
- **Classement :** un livre proposé par plusieurs graines remonte, égalités : somme des scores F1, puis id.
- **Popularité :** F1 pénalise déjà best-sellers et gros lecteurs, F3 affiche la part de ses résultats dans le top 100 et leur popularité médiane.
- **Explication :** chaque recommandation cite ses graines et la raison donnée par F1.
- **Historique court :** --truncate k ne garde que les k premières notes, sans note ≥ 4, F3 le signale et ne recommande rien.

F4 - Tracer un lien entre deux livres

Voisins
≥ 3 lecteurs aiment les deux, lift ≥ 1,5

→

Chemins
direct, puis A → X → B, puis A → X → Y → B

→

Classement
moins d'étapes, puis maillon le plus faible

- **Lift :** lecteurs communs ÷ lecteurs communs attendus par hasard. 1,5 = 50 % de plus que le hasard.
- **Carrefours :** les 100 livres les plus aimés ne servent pas d'intermédiaire et les 5 % de lecteurs les plus actifs sont ignorés, sinon tout serait relié par un best-seller.
- **3 étapes :** cherchées seulement s'il manque des chemins courts, depuis les 20 meilleurs voisins de A.
- **Preuves :** pour chaque étape, les lecteurs communs et leurs notes, et les tags communs en contexte.
- **Pas de lien :** F4 dit pourquoi : aucun voisin convaincant, ou passage obligé par un carrefour.

F5 - Relier deux centres d'intérêt

Groupes
lecteurs qui aiment chacun des deux tags

→

Recoupement
lecteurs dans les deux groupes, vs hasard

→

Livres pont
livres aimés par les deux groupes

- **Groupe d'un tag :** lecteurs qui ont aimé au moins 3 livres portant ce tag, et plus que la moyenne.
- **Force du lien :** lift du recoupement des deux groupes, sous ×1,5, le lien est jugé faible.
- **Livres pont :** aimés par au moins 3 lecteurs de chaque groupe, plus que par l'ensemble des lecteurs, classés par la plus petite des deux parts.
- **Robustesse :** tout est recalculé sans les livres très populaires ni les lecteurs très actifs : le lien tient ou s'effondre.
- **Contrôle :** groupes recalculés sans les livres qui portent les deux tags.

F6 - Mise à jour idempotente

Identité d'une note

Le couple (user_id, book_id), protégé par un index unique.

Conflit de score

La note importée remplace l'ancienne : la plus récente dans l'ordre des fichiers.

Idempotence

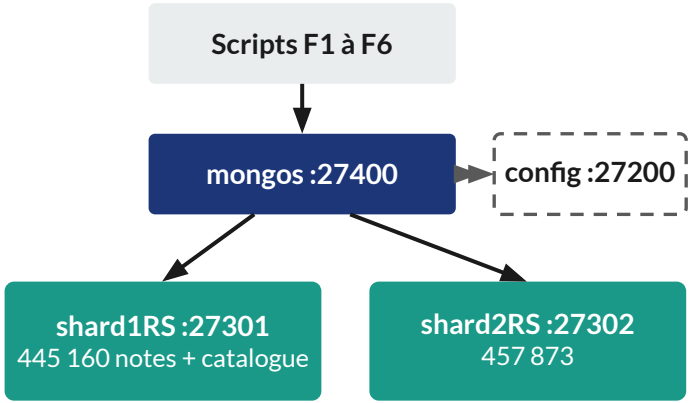
UpdateOne + upsert : 2e import = 0 note insérée, 0 modifiée.

Sur le cluster

Un shard injoignable : rien n'est écrit.

Lot = 10 % de la sélection, mis de côté avant la construction · **1,19 s** d'import, F1 à F5 lisent les notes à la demande et reflètent l'import aussitôt.

Cluster : 2 shards, clé user_id



Notes et listes réparties par {user_id: "hashed"}, catalogue non réparti.

Requête	Shards	Local / échangé	Temps
Notes du lecteur 20000	1	91 clés lues, 91 notes	1 ms
Qui a aimé le livre 1	2	1 630 + 1 730 clés, fusion	5 ms
Livre 1 → lecteurs → livres	2	32 Ko envoyés, comptes partiels, 2e $group	156 ms

Vérification automatique (compare.py) : **26 / 26** sorties identiques entre local et les deux placements, lecteurs et livres de shards différents compris.

Placement alternatif et panne

Placement par livre

- Les lecteurs d'un livre sont sur un seul shard : le parcours passe de 156 à 136 ms.
- Mais l'historique d'un lecteur est coupé en deux.
- En panne, tous les lecteurs perdent une partie de leur historique.
- Nous gardons donc le placement par lecteur.

Perte d'un shard (shard2RS arrêté)

- availability.py vérifie avant le calcul où sont les données utiles.
- F2 répond si les deux lecteurs sont sur shard1RS, sinon refus explicite.
- F3 échoue explicitement, F6 n'écrit rien.
- Jamais de résultat partiel présenté comme complet.

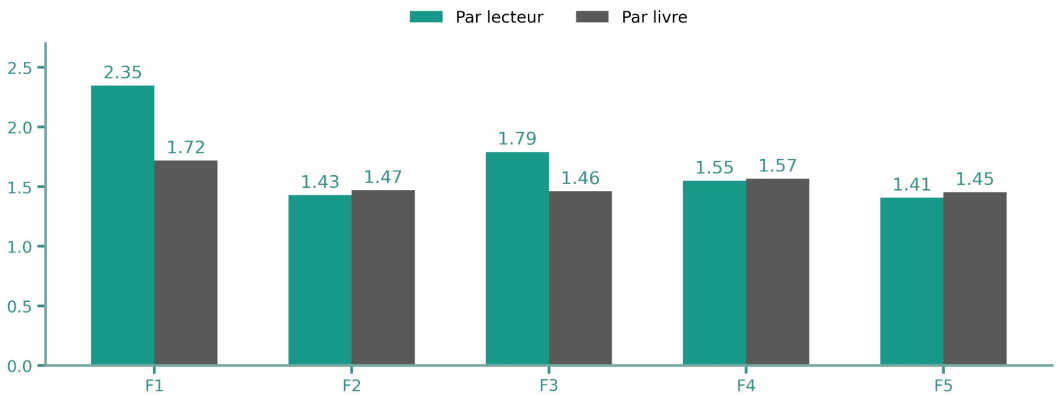
E1 · Passage à l'échelle

Ensemble	1	2	3
Notes	250 000	500 000	903 033
Lecteurs	10 000	10 000	10 000
Livres notés	9 841	9 966	9 983
Stockage	70,5 Mo	88,2 Mo	126,9 Mo
Import	23,8 s	30,0 s	38,7 s

Médiane ×5	1	2	3
F1	2,75 s	3,43 s	4,31 s
F2	169 ms	224 ms	299 ms
F3	24,2 s	23,5 s	26,6 s
F4	38 ms	105 ms	246 ms
F5	1,48 s	2,38 s	3,61 s

Notes ×3,6 : F4 ×6,5, F5 ×2,4, F1 ×1,6. **F3 reste vers 25 s** : ses 10 appels à F1 dominent.

E2 · Local contre distribué



Rapport des temps médians cumulés sur 5 répétitions (référence : local = 1)

Le local est le plus rapide partout.

Le partitionnement par livre gagne pour F1 et F3.

Le distribué ajoute un passage par mongos et un 2e regroupement, sur le même CPU, la même RAM et le même disque.

E3 · Pertinence des recommandations

Historique	Méthode	Recall@10	Prec.@10
Complet	**BookScope**	**0,090**	**0,090**
Complet	Popularité	0,027	0,010
Court (k = 5)	**BookScope**	0,023	0,015
Court (k = 5)	Popularité	0,027	0,010

Évaluation sur les 10 % mis de côté avant toute construction · 20 lecteurs · K = 10.

×3,4 de rappel face à la popularité, historique complet

Démonstration

Bilan

Acquis

- 26 / 26 comparaisons local = cluster
- Chaque lien remonte à ses enregistrements
- F3 : 0 livre du top 100, popularité ÷ 11
- Index couvrants : −39 à −63 %
- Panne : échec explicite, jamais de réponse partielle

Limites et suite

- Un seul poste, shards à un membre
- F3 lent (10 × F1) : E3 sur 20 lecteurs
- Seuils fixés à la main
- Suite : voisins précalculés tenus à jour par F6, vrais replica sets